ShopSphere — SQL Database Assignment Report

Objective

The objective is to design and query an e-commerce database using advanced SQL techniques. The database supports customers, categories, products, orders, order items, payments, and reviews, with PostgreSQL as the recommended database system.

Database Design

The database contains seven core tables: customers, categories, products, orders, order_items, payments, and reviews. Primary keys uniquely identify records, while foreign keys establish relationships between related tables.

Main Tables and Relationships

Table	Purpose	Key Relationship
customers	Stores customer information	customer_id
categories	Stores product categories and parent-child categories	parent_category_id
products	Stores product details, prices and stock	category_id → categories
orders	Stores customer orders and status	customer_id → customers
order_items	Stores products included in orders	order_id → orders; product_id → products
payments	Stores payment details	order_id → orders
reviews	Stores customer product reviews	customer_id → customers; product_id → products

SQL Query Coverage

The assignment contains 120 SQL queries covering basic retrieval, aggregate functions, GROUP BY and HAVING, joins, subqueries, correlated subqueries, CASE expressions, CTEs, recursive CTEs, views, materialized views, indexes, stored procedures, functions, window functions, date/time operations, string functions, NULL handling, set operations, transactions, and advanced business queries.

Performance Optimization Case Study

The case study assumes an orders table containing 1,000,000 rows. The frequently executed query filters by customer_id, order_status and order_date. A composite index on (customer_id, order_status, order_date) is appropriate because the query uses equality filters on customer_id and order_status followed by a date range filter. EXPLAIN ANALYZE can be used to inspect whether PostgreSQL uses the index and to review execution time and the execution plan.

Potential disadvantages of the index include additional storage, maintenance overhead, and possible performance overhead for INSERT, UPDATE and DELETE operations.

View vs Materialized View Case Study

For a dashboard opened hundreds of times per hour where information only needs to update once every hour, a materialized view is appropriate. It stores the result of the reporting query and can reduce repeated aggregation work. The trade-off is that the stored information can become stale until the materialized view is refreshed.

The executive dashboard includes monthly revenue, total orders, total customers, top-selling-product information, total products sold and average order value. The materialized view can be refreshed hourly.

Customer Performance Report

The final Customer Performance Report displays customer_id, customer_name, city, registration_date, total_orders, completed_orders, cancelled_orders, total_products_purchased, total_spent, average_order_value, last_order_date, customer_rank and customer_category.

Customer classification: spending of 20,000 or more is Platinum; 10,000 or more is Gold; 5,000 or more is Silver; otherwise the customer is Regular.

The report uses joins, grouping, aggregate functions, CASE expressions, a CTE and a window function, and is implemented as the view customer_performance_report.

Executive Sales Dashboard

The final materialized view is named executive_sales_dashboard. It contains sales_month, total_revenue, total_orders, total_customers, total_products_sold and average_order_value. An index on sales_month supports month-based filtering, and REFRESH MATERIALIZED VIEW updates the stored results.

Transactions

A transaction groups related database operations into one logical unit. For placing an order, the order is created, order items are inserted, product stock is reduced, payment information is inserted, and the transaction is committed when all operations succeed. If an operation fails, the transaction can be rolled back so that partial changes are not retained.

Views, Procedures, Functions and Indexes

Views provide reusable query definitions. Materialized views store query results for faster repeated reads. Stored procedures perform database operations and can modify data. User-defined functions return calculated values or results. Indexes improve lookup performance for suitable queries but require storage and maintenance.

Documentation Questions — Answers

1. What is a primary key?

A primary key uniquely identifies each row in a table. It cannot contain duplicate values and normally cannot be NULL.

2. What is a foreign key?

A foreign key is a column or set of columns that references a key in another table. It helps maintain referential integrity between related tables.

3. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY and is commonly used with aggregate functions.

4. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows having matching records in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table; unmatched right-side values become NULL.

5. What is a subquery?

A subquery is a query nested inside another SQL query. It can provide values or result sets used by the outer query.

6. What is a correlated subquery?

A correlated subquery depends on a value from the current row of the outer query and is evaluated in relation to that outer row.

7. What is a CTE?

A Common Table Expression, created with WITH, defines a temporary named result set that can be referenced by the main query. It improves readability and helps break complex queries into logical steps.

8. What is a recursive CTE?

A recursive CTE repeatedly processes rows using an anchor query and a recursive query. It is useful for hierarchical data such as parent-child categories.

9. What is a view?

A view is a stored SQL query definition that can be queried like a table. It normally does not store a separate physical copy of the result.

10. What is a materialized view?

A materialized view stores the result of a query physically. It can make repeated reporting queries faster, but its data must be refreshed.

11. Difference between view and materialized view?

A normal view evaluates its underlying query when accessed, while a materialized view stores a result that can be refreshed. Materialized views can improve repeated-read performance but may contain stale data.

12. What is an index?

An index is a database structure that helps PostgreSQL locate rows more efficiently for suitable queries.

13. What are advantages and disadvantages of indexes?

Indexes can improve SELECT performance, especially for filtering, joining and ordering. They require storage and can add overhead to INSERT, UPDATE and DELETE operations.

14. What is a composite index?

A composite index is an index built on multiple columns. Column order matters because it affects which query conditions can efficiently use the index.

15. What is a stored procedure?

A stored procedure is a named database program that can perform a sequence of operations and can modify database data.

16. What is a user-defined function?

A user-defined function is a database function created by the developer to perform reusable calculations or return a value/result.

17. Difference between procedure and function?

A procedure is generally used to perform operations and is invoked with CALL. A function returns a value or result and can be used in SQL expressions.

18. What is a window function?

A window function performs calculations across related rows without collapsing them into one row per group. Examples include RANK, ROW_NUMBER and LAG.

19. What is RANK()?

RANK assigns ranking values according to an ORDER BY expression. Tied rows receive the same rank, and subsequent ranks can contain gaps.

20. What is ROW_NUMBER()?

ROW_NUMBER assigns a unique sequential number to each row within the specified ordering and partition.

21. What is a transaction?

A transaction is a logical unit of database work whose operations are committed together or rolled back together.

22. Difference between COMMIT and ROLLBACK?

COMMIT permanently saves the successful transaction changes. ROLLBACK cancels uncommitted changes and returns the database to its previous transaction state.

23. What is EXPLAIN ANALYZE?

EXPLAIN ANALYZE executes a query and reports the execution plan together with runtime information. It is useful for identifying scans, joins and performance bottlenecks.

24. What is normalization?

Normalization is the process of organizing data into related tables to reduce unnecessary duplication and improve data integrity.

25. Why is database design important?

Good database design organizes related information clearly, enforces integrity through keys and constraints, reduces unnecessary duplication, and provides a reliable foundation for querying and maintaining business data.

Conclusion

The ShopSphere database demonstrates practical PostgreSQL database design and advanced SQL querying. The implementation covers relational design, analytical queries, joins, subqueries, CTEs, views, materialized views, indexing, procedures, functions, transactions, window functions and business reporting. The final customer report and executive dashboard demonstrate how the database can support operational and management-level analysis.

Submission Checklist

File / Deliverable	Status
01_schema.sql	Prepared
02_sample_data.sql	Prepared
03_queries.sql	Queries 1–120 plus later case-study/report SQL
04_views.sql	Required deliverable
05_materialized_views.sql	Required deliverable
06_indexes.sql	Required deliverable
07_procedures.sql	Required deliverable
08_functions.sql	Required deliverable
09_ctes.sql	Required deliverable
Final report	This document